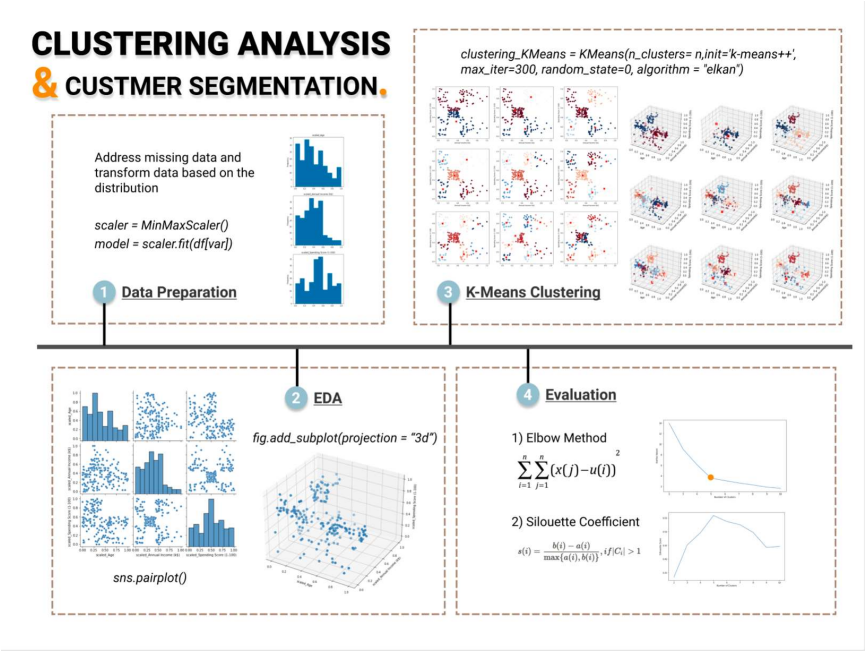
Clustering , Segmentation

- Varun Raste

Segmentation

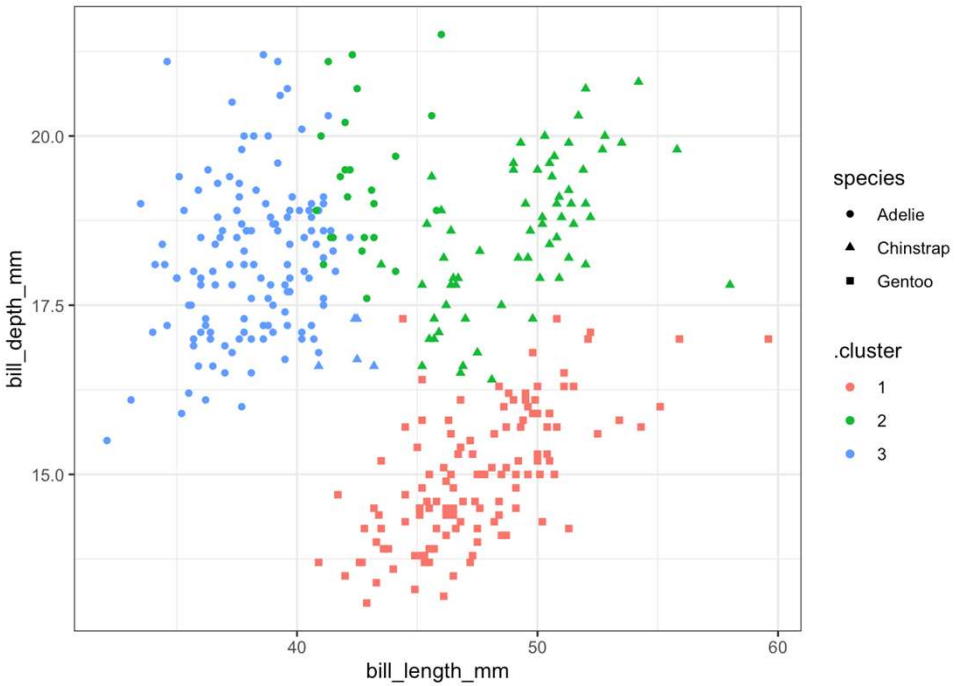
- Market segmentation groups similar customers instead of treating everyone the same.
- It helps businesses tailor marketing, pricing, and product strategies.
- Clustering is used when no target variable exists.
- Patterns emerge naturally from customer behavior.
- Effective segmentation drives better decision-making.



Please note , all Images are taken from internet sources , author doesnt claim right over images*

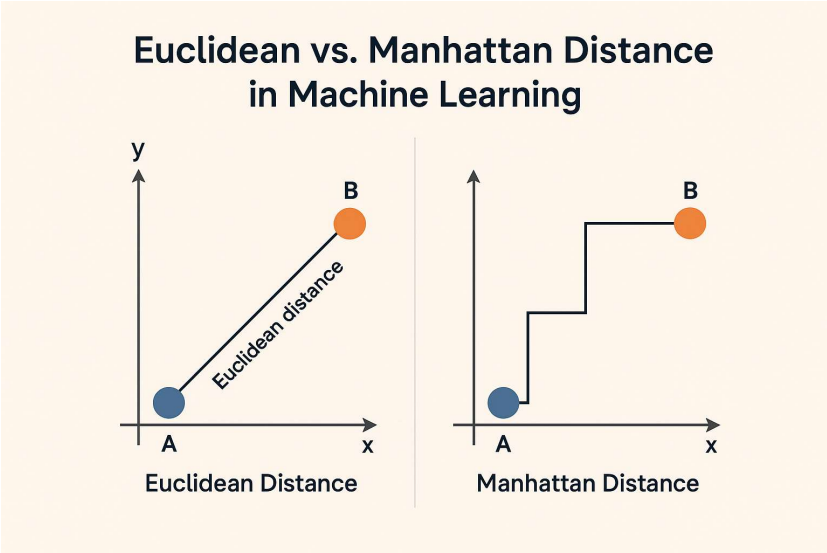
Clustering in Segmentation

- Clustering is an unsupervised learning technique used to group customers based on similarity.
- Unlike classification, clustering works without predefined labels or outcomes.
- Similarity is measured using distance or density between customer features.
- Each cluster captures a distinct behavioral or demographic pattern.
- The objective is to uncover natural groupings hidden in the data.
- These groupings later drive targeted business strategies.



Distance Metrics

- Distance metrics define how similarity between data points is calculated.
- Clustering decisions depend heavily on the chosen distance metric.
- Euclidean distance works well for continuous, scaled numerical features.
- Manhattan distance is useful when differences are absolute and directional.
- Cosine similarity compares orientation rather than magnitude.
- An inappropriate distance metric can distort cluster structure.

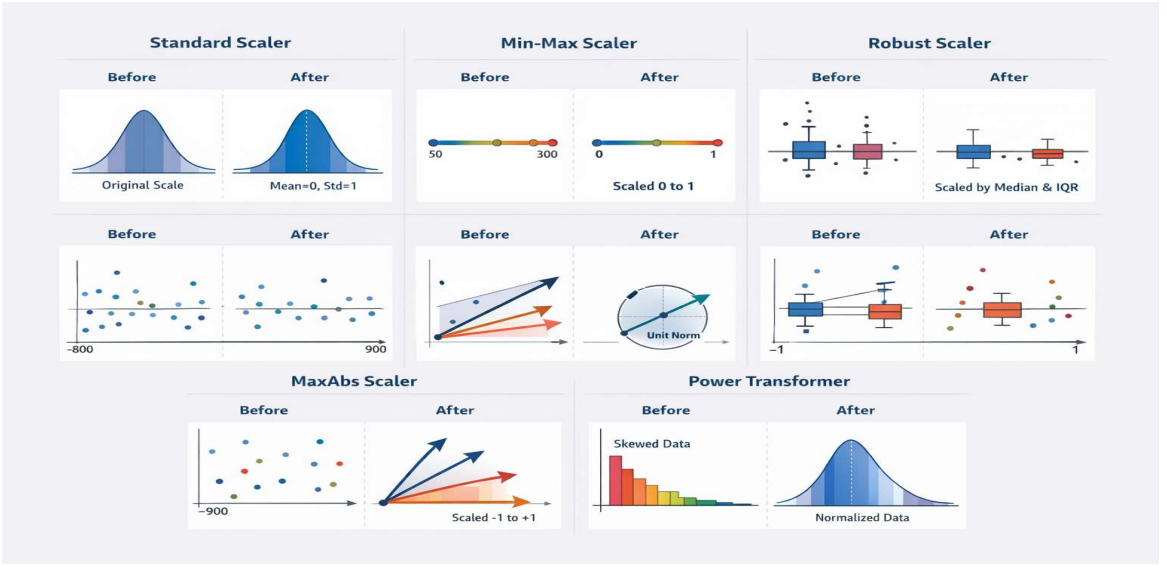


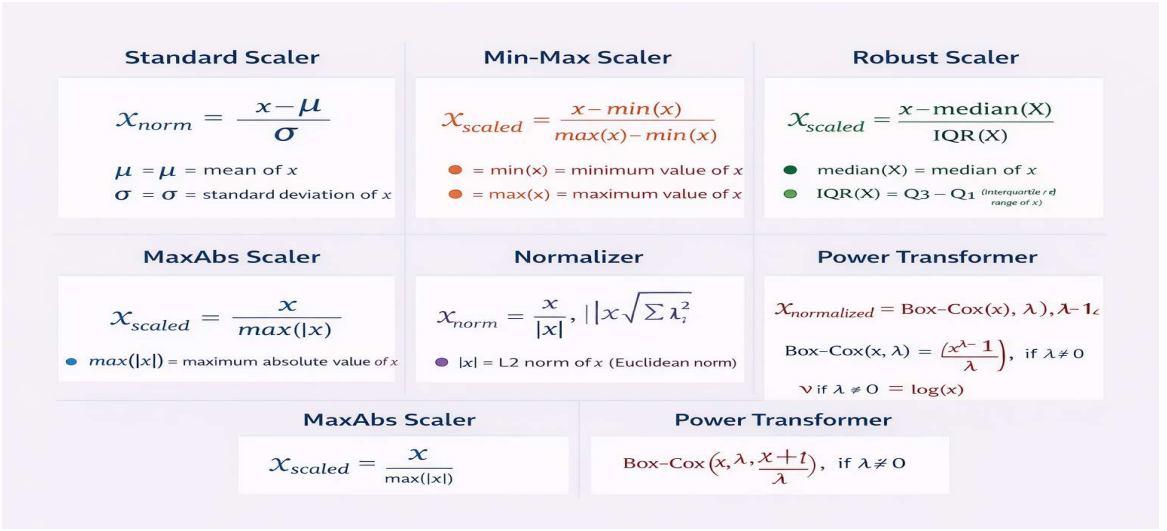
Distance Metrics

Metric	How it Measures Distance	Best Used When	Pros	Cons / Pitfalls	Typical Use Cases
Euclidean	Straight-line distance between two points	Features are continuous and well-scaled	Simple, intuitive, widely supported	Sensitive to scale and outliers	K-Means, customer spend segmentation
Manhattan	Sum of absolute differences across features	Data represents step-wise or grid-like movement	More robust to outliers than Euclidean	Ignores diagonal proximity	Urban distance, logistics, tabular data
Mahalanobis	Distance accounting for feature correlation	Features are correlated	Handles covariance, scale-invariant	Computationally expensive, unstable with small data	Finance risk, anomaly detection
Cosine	Angle between vectors (orientation)	Magnitude is less important than direction	Scale-invariant, great for sparse data	Ignores absolute magnitude	Text clustering, embeddings
Minkowski	Generalized distance (Euclidean & Manhattan special cases)	Flexible distance tuning required	Adjustable behavior via parameter *p*	Harder to interpret	Experimental clustering
Haversine	Great-circle distance on a sphere	Geo-coordinates (lat, long)	Accurate for earth distances	Only for geographic data	Geo-segmentation, delivery zones

Role of Scaling in Clustering

- Distance-based clustering algorithms compute similarity purely using numerical distances between points.
- When features are on different scales, larger-valued features dominate distance calculations.
- As a result, clusters may reflect measurement units instead of true customer behavior patterns.
- StandardScaler rescales features to have zero mean and unit variance, making them comparable.
- MinMaxScaler compresses values into a fixed range while preserving relative spacing.
- The choice of scaler should depend on feature distribution, outliers, and business meaning.



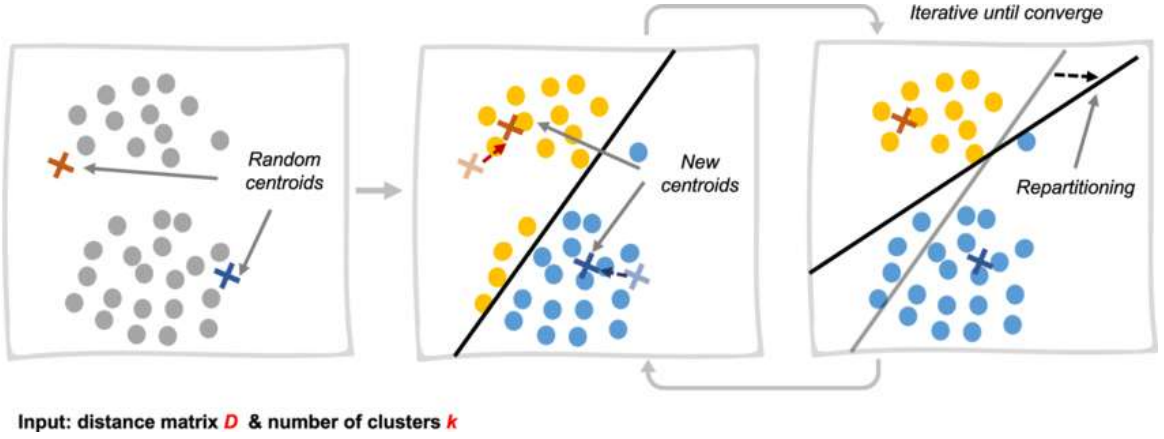


Types of Scalers & Comparison

Scaler	How It Transforms Data	Best Used When	Advantages	Limitations / Risks	Common Use in Clustering
StandardScaler	Centers data to mean 0 and standard deviation 1	Features are approximately normally distributed	Balances feature influence, widely used	Sensitive to outliers	K-Means, hierarchical clustering
MinMaxScaler	Scales values to a fixed range (usually 0–1)	Feature bounds are meaningful	Preserves relative distances	Extremely sensitive to outliers	Customer behavior segmentation
RobustScaler	Uses median and interquartile range	Data contains strong outliers	Resistant to outliers	Loses distribution shape	Noisy real-world datasets
MaxAbsScaler	Scales by maximum absolute value	Data is already centered around zero	Keeps sparsity intact	Does not center data	Sparse matrices, embeddings
Normalizer	Scales each row to unit norm	Direction matters more than magnitude	Scale-invariant per sample	Destroys absolute magnitude	Cosine-based clustering
PowerTransformer	Applies power transformation to normalize data	Skewed distributions	Makes data more Gaussian	Harder to interpret	Advanced clustering pipelines

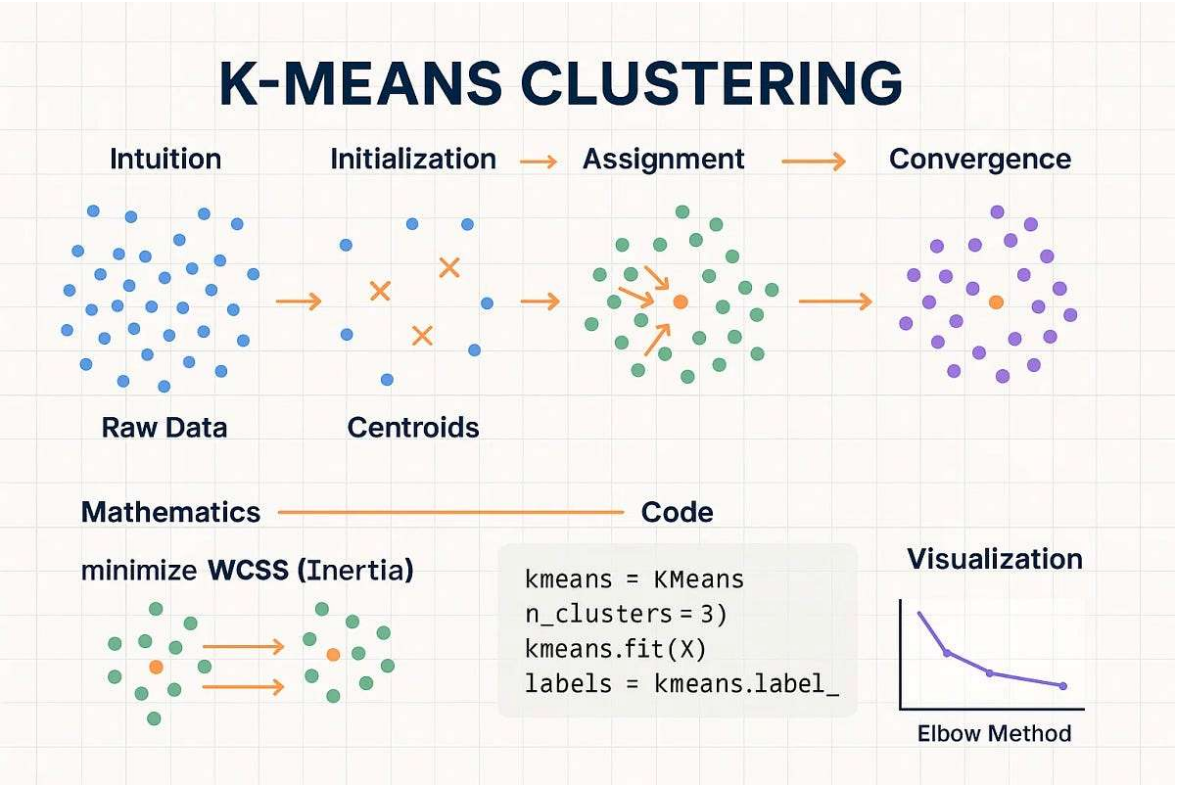
Kmeans Clustering

- K-Means is an unsupervised clustering algorithm used to group similar data points.
- It partitions the dataset into K distinct clusters based on distance.
- Each cluster is represented by a centroid, which acts as the cluster center.
- Data points are assigned to the nearest centroid during each iteration.
- The algorithm aims to minimize within-cluster variance.
- K-Means works best with compact and well-separated clusters.



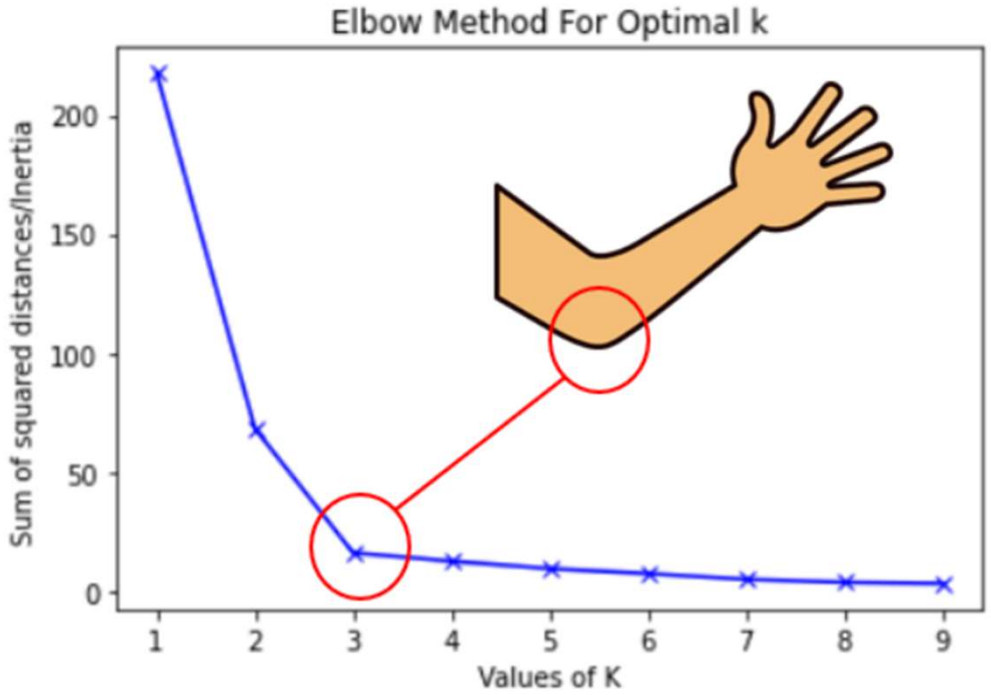
Step By Step Guide

- The algorithm begins by randomly selecting K initial centroids.
- Each data point is assigned to the nearest centroid based on distance.
- New centroids are recalculated as the mean of assigned points.
- Assignment and update steps repeat iteratively.
- The process stops when centroids no longer change significantly.
- The final output is a stable set of K clusters.



How to Choose K

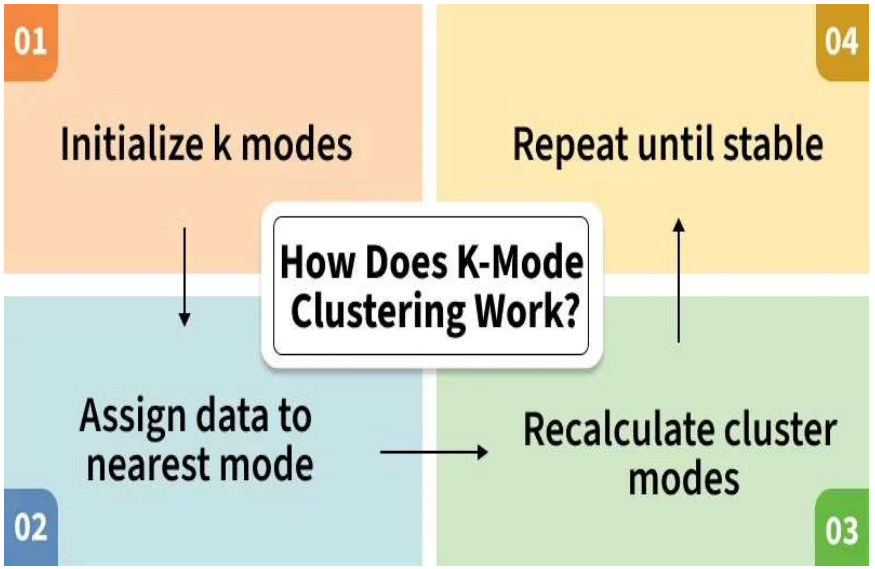
- K-Means requires the number of clusters (K) to be specified beforehand.
- The Elbow Method helps identify a suitable K using within-cluster variance.
- We plot K against the sum of squared distances within clusters.
- As K increases, variance decreases but at a slower rate.
- The “elbow point” marks diminishing returns in variance reduction.
- This point suggests a balance between simplicity and performance.



Line plot between K and inertia

Kmodes Clustering

- K-Modes is a clustering algorithm designed for categorical data.
- It replaces numerical distance with mode-based similarity.
- Cluster centers are represented by modes, not means.
- Distance is computed using simple matching dissimilarity.
- Ideal when numerical encoding would distort meaning.
- Works directly on categorical variables without encoding.
- Faster and simpler than mixed-data approaches.
- Avoids artificial distance created by one-hot encoding.
- Not suitable for numerical-only datasets.
- Sensitive to initialization and choice of K.

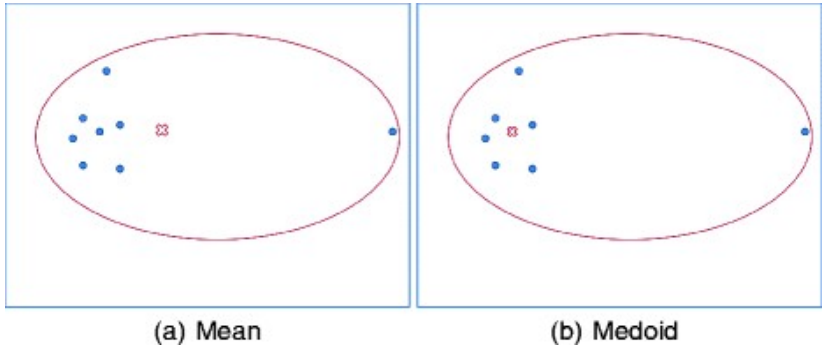


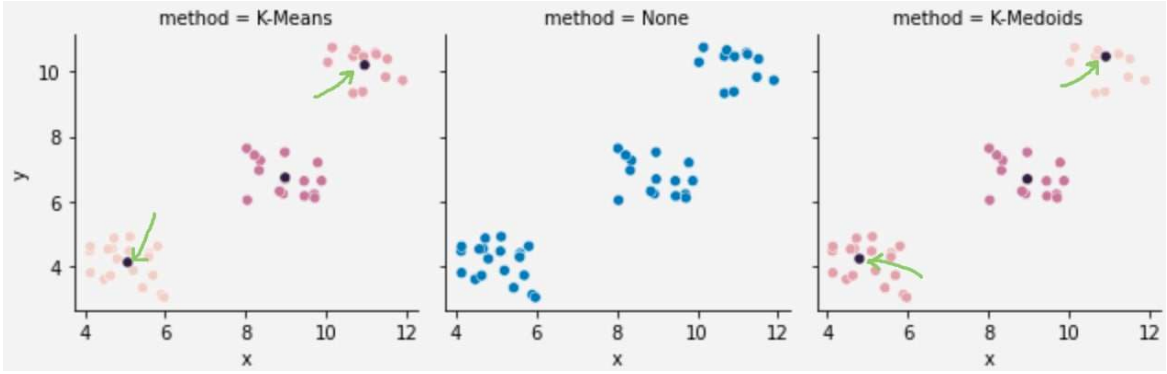
K-Prototype Clustering

- K-Prototypes handles mixed data (numerical + categorical).
- It combines Euclidean distance and categorical mismatch.
- Cluster centers contain both means and modes.
- A weighting factor balances numeric vs categorical influence.
- Designed for real-world customer datasets.
- Avoids forced encoding of categorical variables.
- Preserves meaning across mixed feature types.
- Requires tuning of the categorical weight parameter.
- Still needs K to be predefined.
- Less interpretable than pure K-Means or K-Modes.

K-Medoids Clustering

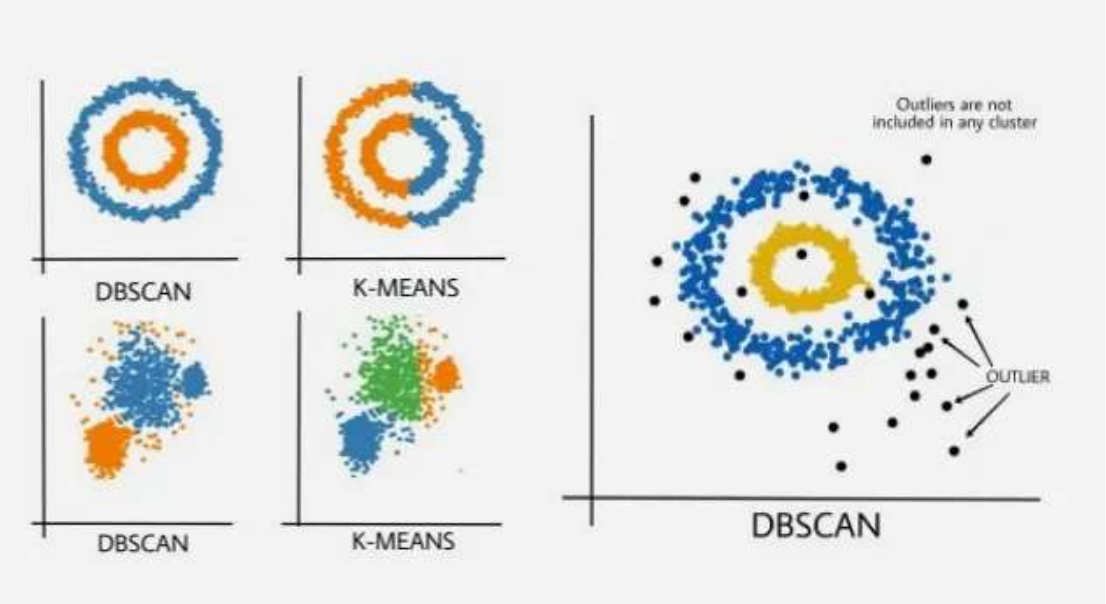
- K-Medoids selects actual data points as cluster centers.
- Medoids are more robust than centroids.
- Distance can be any metric, not just Euclidean.
- Less sensitive to extreme values and noise.
- Often implemented using the PAM algorithm.





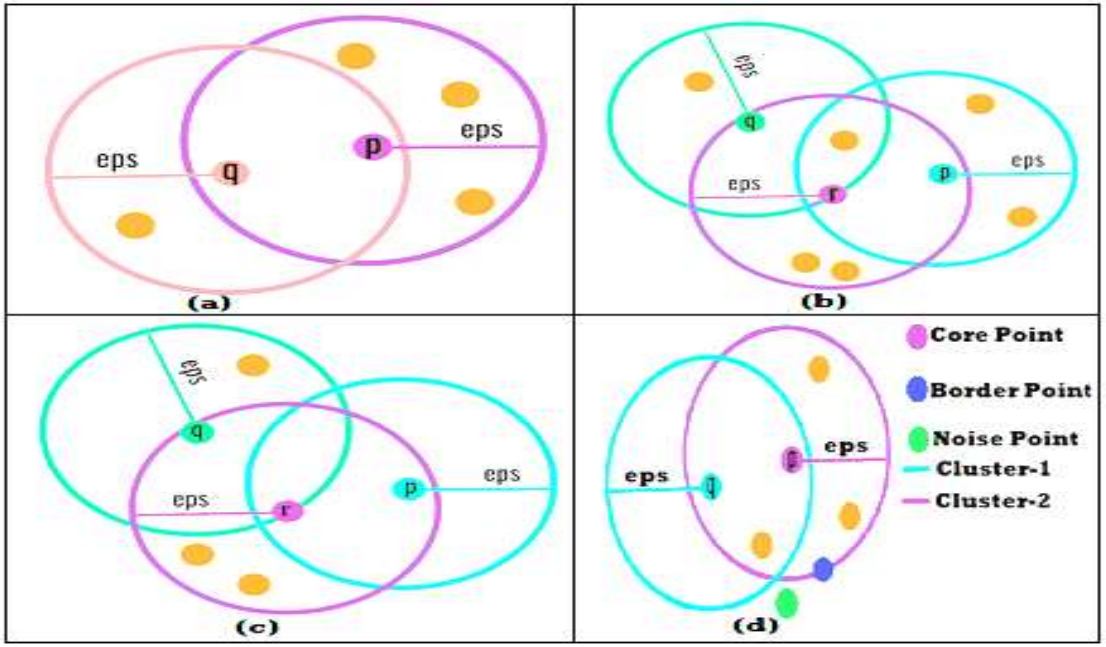
DBScan Clustering

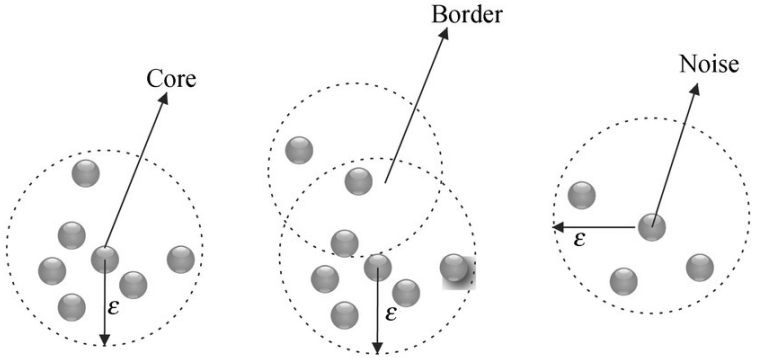
- DBSCAN groups data points based on areas of high point density.
- A cluster is formed when enough points exist within a local neighborhood.
- **Epsilon (ε)** defines the radius used to search neighboring points.
- **MinPts** specifies the minimum number of points required to form a dense region.
- Points are classified as **core**, **border**, or **noise** points.
- The algorithm expands clusters by connecting dense regions together.



Step By Step Guide

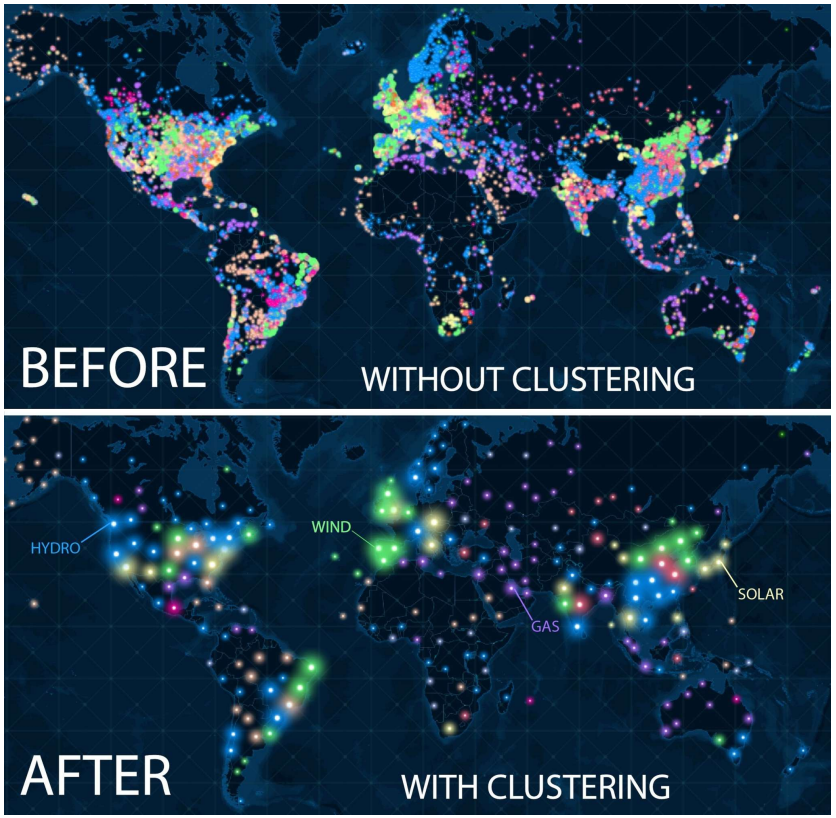
- Select an unvisited data point from the dataset.
- Find all neighboring points within distance **ε (epsilon)**.
- If neighbors ≥ **MinPts**, mark the point as a **core point**.
- Expand the cluster by recursively visiting density-reachable points.
- Assign **border points** that lie within ε of a core point.
- Label isolated points as **noise** if density criteria are not met.





Geo-Spatial Clustering

- Geo-spatial clustering groups data points based on physical location.
- Latitude and longitude act as primary features for clustering.
- Distance represents real-world proximity, not abstract similarity.
- Useful for identifying regional patterns and spatial hotspots.
- Often combined with demographic or behavioral attributes.
- Euclidean distance can be misleading on Earth’s surface.
- Haversine distance is preferred for latitude–longitude data.
- Proper distance choice improves cluster realism.



Silhouette Score

- Measures how close a point is to its own cluster.
- Compares cohesion vs separation simultaneously.
- Values range from -1 to +1.
- Higher values indicate better-defined clusters.

Davies–Bouldin Index

- Evaluates average similarity between clusters.
- Considers cluster compactness and separation.
- Sensitive to overlapping clusters.
- Lower values indicate better clustering.

Calinski–Harabasz Index

- Measures ratio of between-cluster variance to within-cluster variance.
- Rewards well-separated and compact clusters.
- Works well for K-Means-like algorithms.
- Higher values suggest better-defined clusters.

Inertia (Within-Cluster SSE)

- Captures how tightly points are packed within clusters.
- Directly minimized by K-Means.
- Always decreases as K increases.
- Best used with Elbow Method.

Comparison of Clustering Algorithms

Scenario / Requirement	Recommended Algorithm	Why This Works	Typical Applications
Numerical data only	**K-Means**	Fast, scalable, distance-based	Spend segmentation, usage patterns
Categorical data only	**K-Modes**	Uses mode-based similarity	Survey analysis, customer attributes
Mixed numerical + categorical	**K-Prototypes**	Combines Euclidean + categorical distance	Customer profiling, CRM segmentation
Data contains strong outliers	**K-Medoids**	Robust center selection	Finance risk, healthcare data
Number of clusters unknown	**DBSCAN**	Density-based, auto cluster discovery	Anomaly detection, spatial data
Irregular cluster shapes	**DBSCAN**	No spherical cluster assumption	Geo-spatial analysis
Large dataset, performance critical	**K-Means**	Computationally efficient	Large-scale marketing data
High interpretability needed	**K-Medoids**	Centers are real data points	Business reporting
Noise & anomaly detection	**DBSCAN**	Explicit noise labeling	Fraud detection
Location-based segmentation	**DBSCAN / K-Medoids**	Distance & density aware	Store placement, logistics

Important Parameters / hyper-parameters

Algorithm	Hyperparameter	What It Controls	Typical / Ideal Starting Values	Practical Notes
K-Means	`n_clusters (K)`	Number of clusters	3–10 (start with Elbow)	Use Elbow + Silhouette
	`init`	Centroid initialization	`k-means++`	Faster convergence, better stability
	`max_iter`	Max optimization steps	300	Increase only if not converging
	`n_init`	Number of initializations	10–20	Higher = more stable results
	`random_state`	Reproducibility	Fixed integer	Mandatory for experiments
K-Modes	`n_clusters`	Number of clusters	3–10	Similar logic as K-Means
	`init`	Mode initialization	`Huang`	Default and reliable
	`max_iter`	Iterations	100	Usually converges fast
	`n_init`	Initializations	5–10	Helps avoid poor local minima
K-Prototypes	`n_clusters`	Number of clusters	3–10	Use business intuition + metrics
	`gamma`	Weight for categorical distance	0.1–1.0 (tune)	Most critical parameter
	`init`	Prototype initialization	`Huang`	Stable for mixed data
	`max_iter`	Iterations	100	Increase for complex data
K-Medoids	`n_clusters`	Number of clusters	3–10	Same selection logic as K-Means
	`metric`	Distance function	`euclidean`, `manhattan`	Choose based on data
	`method`	Optimization approach	`PAM`	Standard and interpretable
	`max_iter`	Iterations	300	More costly than K-Means
DBSCAN	`eps (ε)`	Neighborhood radius	Use k-distance plot	Most sensitive parameter
	`min_samples`	Min points to form cluster	2×dimensions (rule of thumb)	Controls noise vs clusters
	`metric`	Distance measure	`euclidean`, `haversine`	Geo data → Haversine
	`leaf_size`	Tree optimization	30–50	Performance tuning only